understanding how data is stored and retrieved, which we'll start to unfold in the next section.

8.3 Delving into semantic search and document indexing

Document indexing transforms a document's information to be more easily recovered. How the index will be queried or searched also plays a factor, whether searching for a particular set of words or wanting to match phrase for phrase.

A *semantic* search is a search for content that matches the searched phrase by words and meaning. The ability to search by meaning, semantically, is potent and worth investigating in some detail. In the next section, we look at how vector similarity search can lay the framework for semantic search.

8.3.1 Applying vector similarity search

Let's look now at how a document can be transformed into a *semantic vector,* or a representation of text that can then be used to perform distance or similarity matching. There are numerous ways to convert text into a semantic vector, so we'll look at a simple one.

Open the `chapter_08` folder in a new Visual Studio Code (VS Code) workspace. Create a new environment and `pip install` the `requirements.txt` file for all the chapter dependencies. If you need help setting up a new Python environment, consult appendix B.

Now open the `document_vector_similarity.py` file in VS Code, and review the top section in listing 8.1. This example uses Term Frequency–Inverse Document Frequency (TF–IDF). This numerical statistic reflects how important a word is to a document in a collection or set of documents by increasing proportionally to the number of times a word appears in the document and offset by the frequency of the word in the document set. TF–IDF is a classic measure of understanding one document's importance within a set of documents.